

Evidencias modulo 1

The screenshot shows the Python Institute logo in the top left corner. The page title is 'PE1 -- Module 1 Quiz'. A progress bar at the top indicates 'Progreso (100%)'. The main content area displays 'Tu puntaje: 10/10' and a large '100%' score. Below this, it says 'Congratulations, you've passed the quiz!'. A section titled 'ANÁLISIS DE LA SECCIÓN' shows 'PE1 -- Module 1 Quiz' with a score of '100%'. At the bottom, there are two buttons: 'Volver a presentar la prueba' and 'Revisar prueba'.

ANÁLISIS DE LA SECCIÓN	
PE1 -- Module 1 Quiz	100%

Python INSTITUTE

FP1: Examen Módulo 1

En línea

Progreso (100%)

≡

Tu puntaje: 10/10

100%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 1 Test100%

Volver a presentar la prueba

Revisar prueba

Modulo 2

Python INSTITUTE

MODULE (37%)

SECTION (10%)

« 2.1.1.1 Tu primer programa »

☰

🔍

⚙️

▶

📄

🔗

📁

🔄

Sandbox

¡Hola, Mundo!

Es hora de comenzar a escribir **código real y funcional en Python**. Por el momento será muy sencillo.

Como se muestran algunos conceptos y términos fundamentales, estos fragmentos de código no serán complejos ni difíciles.

Ejecuta el código en la ventana del editor a la derecha. Si todo sale bien, verás la **línea de texto** en la ventana de consola.

Como alternativa, inicia IDLE, crea un nuevo archivo fuente de Python, coloca este código, nombra el archivo y guárdalo. Ahora ejecútalo. Si todo sale bien, verás el texto contenido entre comillas en la ventana de la consola IDLE. El código que has ejecutado debería parecerte familiar. Viste algo muy similar cuando te guiamos a través de la configuración del entorno IDLE.

Ahora dedicaremos un poco de tiempo para mostrarte y explicarte lo que estás viendo y por que se ve así.

Como puedes ver, el primer programa consta de las siguientes partes:

- La palabra `print`.
- Un paréntesis de apertura.
- Una comilla.

```
1 print("¡Hola, Mundo!")
2
3
```

Console >_

¡Hola, Mundo!

PythonINSTITUTE

« 2.1.1.6 LABORATORIO: La función print() »

MODULE (10%)

SECTION (33%)

Sandbox

Objetivos

- Familiarizarse con la función `print()` y sus capacidades de formato.
- Experimentar con el código de Python.

Escenario

El comando `print()`, el cual es una de las directivas más sencillas de Python, simplemente imprime una línea de texto en la pantalla.

En tu primer laboratorio:

- Utiliza la función `print()` para imprimir la línea `"¡Hola, Mundo!"` en la pantalla.
- Una vez hecho esto, utiliza la función `print()` nuevamente, pero esta vez imprime tu nombre.
- Elimina las comillas dobles y ejecuta el código. Observa la reacción de Python. ¿Qué tipo de error se produce?
- Luego, elimina los paréntesis, vuelve a poner las comillas dobles y vuelve a ejecutar el código. ¿Qué tipo de error se produce esta vez?
- Experimenta tanto como puedas. Cambia las comillas dobles a comillas simples, utiliza múltiples funciones `print()` en la misma línea y luego en líneas diferentes. Observa que es lo que ocurre.

```
1 print("Carlos Antonio")
2 print('Carlos Antonio') print('Holaaaa')
3 print('Arriba las Chivas')
4 print('Mi primer programa en python')]
```

Console >_

```
¡Hola, Mundo!
¡Hola, Mundo!
Carlos Antonio
Carlos Antonio
File "main.py", line 2
  print(Carlos Antonio)
      ^
SyntaxError: invalid syntax
File "main.py", line 2
  print"Carlos Antonio"
    ^
SyntaxError: invalid syntax
File "main.py", line 2
  print('Carlos Antonio') print('Holaaaa')
                        ^
SyntaxError: invalid syntax
```

PythonINSTITUTE

« 2.1.1.8 Tu primer programa »

MODULE (13%)

SECTION (43%)

Sandbox

construcciones complejas.

Vamos a expandir el código un poco, puedes verlo en el editor. Ejecútalo y observa lo que aparece en la consola.

Tu consola Python ahora debería verse así:

La Witsi Witsi Araña subió a su telaraña.
Vino la lluvia y se la llevó.

salida

Esta es una buena oportunidad para hacer algunas observaciones:

- El programa **invoca la función** `print()` **dos veces**, como puedes ver hay dos líneas separadas en la consola: esto significa que `print()` comienza su salida desde una nueva línea cada vez que comienza su ejecución. Puedes cambiar este comportamiento, pero también puedes usarlo a tu favor.
- Cada invocación de `print()` contiene una cadena diferente, como su argumento y el contenido de la consola lo reflejan, esto significa que **las instrucciones en el código se ejecutan en el mismo orden** en que se colocaron en el archivo fuente; no se ejecuta la siguiente instrucción hasta que se complete la anterior (hay algunas excepciones a esta regla, pero puedes ignorarlas por ahora).

```
1 print("La Witsi Witsi Araña subió a su telaraña.")
2 print("Vino la lluvia y se la llevó.")
3
```

Console >_

```
La Witsi Witsi Araña subió a su telaraña.
Vino la lluvia y se la llevó.
```

PythonINSTITUTE

« 2.1.1.9 Tu primer programa »

MODULE (14%)

SECTION (48%)

Sandbox

La función print() - instrucciones

Hemos cambiado un poco el ejemplo: hemos agregado una invocación **vacía** de la función `print()`, la llamamos vacía porque no hemos agregado ningún argumento a la función.

Lo puedes ver en la ventana del editor. Ejecuta el código.

¿Qué ocurre?

Si todo sale bien, deberías ver algo como esto:

La Witsi Witsi Araña subió a su telaraña.
Vino la lluvia y se la llevó.

salida

Como puedes ver, la invocación de `print()` vacía no esta tan vacía como se esperaba - genera una línea vacía (esta interpretación también es correcta) su salida es solo una nueva línea.

Esta no es la única forma de producir una **nueva línea** en la consola de salida. Enseguida mostraremos otra manera.

```
1 print("La Witsi Witsi Araña subió a su telaraña.")
2 print()
3 print("Vino la lluvia y se la llevó.")
4
```

Console >_

```
La Witsi Witsi Araña subió a su telaraña.

Vino la lluvia y se la llevó.
```

Python INSTITUTE

« 2.1.1.10 Tu primer programa »

HOME

MODULE (15%)

SECTION (52%)

≡

🔍

⚙️

▶

🔗

🔄

👤

📁

🔄

Sandbox

En otras palabras, la barra invertida no significa nada, sino que es solo un tipo de anuncio, de que el siguiente carácter después de la barra invertida también tiene un significado diferente.

La letra `n` colocada después de la barra invertida proviene de la palabra *newline* (nueva línea).

Tanto la barra diagonal inversa como la *n* forman un símbolo especial denominado **carácter de nueva línea** (newline character), que incita a la consola a iniciar una **nueva línea de salida**.

Ejecuta el código. La consola ahora debería verse así:

```
La Witsi Witsi Araña
subió a su telaraña.

Vino la lluvia
y se la llevó.
```

Como se puede observar, aparecen dos nuevas líneas en la canción infantil, en los lugares donde se ha utilizado `\n`.

```
1 print("La Witsi Witsi Araña\nsubió a su telaraña.\n")
2 print()
3 print("Vino la lluvia\ny se la llevó.")
4
```

Console >_

La Witsi Witsi Araña
subió a su telaraña.

Vino la lluvia
y se la llevó.

Python INSTITUTE

« 2.1.1.11 Tu primer programa »

HOME

MODULE (17%)

SECTION (57%)

≡

🔍

⚙️

▶

🔗

🔄

👤

📁

🔄

Sandbox

La función `print()` - los caracteres de escape y nueva línea

El utilizar la diagonal invertida tiene dos características importantes:

1. Si deseas colocar solo una barra invertida dentro de una cadena, no olvides su naturaleza de escape: tienes que duplicarla, por ejemplo, la siguiente invocación causará un error:

```
print("\")
```

Mientras que esta no lo hará:

```
print("\\")
```

2. No todos los pares de escape (la diagonal invertida junto con otro carácter) significan algo.

Experimenta con el código en el editor, ejecútalo y observa lo que sucede.

```
1 print("La Witsi Witsi Araña\nsubió a su telaraña.")
2 print()
3 print("Vino la lluvia\ny se la llevó.")
4
```

Console >_

La Witsi Witsi Araña
subió a su telaraña.

Vino la lluvia
y se la llevó.

Python INSTITUTE

« 2.1.1.12 Tu primer programa »

HOME

MODULE (18%)

SECTION (62%)

≡

🔍

⚙️

▶

🔗

🔄

👤

📁

🔄

Sandbox

Observa la ventana del editor. Esto es lo que vamos a probar ahora:

```
print("La Witsi Witsi Araña", "subió", "a su telaraña.")
```

Hay una invocación de la función `print()` pero contiene **tres argumentos**. Todos ellos son cadenas.

Los argumentos están **separados por comas**. Se han rodeado de espacios para hacerlos más visibles, pero no es realmente necesario y no se hará más.

En este caso, las comas que separan los argumentos desempeñan un papel completamente diferente a la coma dentro de la cadena. El primero es una parte de la sintaxis de Python, el segundo está destinado a mostrarse en la consola.

Si vuelves a observar el código, verás que no hay espacios dentro de las cadenas.

Ejecuta el código y observa lo que pasa.

La consola ahora debería mostrar el siguiente texto:

```
La Witsi Witsi Araña subió a su telaraña.
```

Los espacios, removidos de las cadenas, han vuelto a aparecer. ¿Puedes explicar por qué?

```
1 print("La Witsi Witsi Araña", "subió", "a su telaraña.")
```

Console >_

La Witsi Witsi Araña subió a su telaraña.

Herramienta Recortes

Captura de pantalla copiada en el portapapeles
Guardado automáticamente en la carpeta de capturas de pantalla.

Marcado y uso compartido

MODULE (15%)

SECTION (87%)



Sandbox

La función `print()` - la manera posicional de pasar argumentos

Ahora que sabes un poco acerca de la función `print()` y como personalizarla, te mostraremos como cambiarla.

Deberías de poder predecir la salida sin ejecutar el código en el editor.

La forma en que pasamos los argumentos a la función `print()` es la más común en Python, y se denomina **manera posicional** (este nombre proviene del hecho de que el significado del argumento está dictado por su posición, por ejemplo, el segundo argumento se emitirá después del primero, y no al revés).

Ejecuta el código y verifica si la salida coincide con tus predicciones.

```
1 print("Mi nombre es", "Python.")
2 print("Monty Python.")
3
```

Console >_

```
Mi nombre es Python.
Monty Python.
```

MODULE (21%)

SECTION (71%)



Sandbox

La función `print()` - los argumentos de palabra clave

Python ofrece otro mecanismo para transmitir o pasar los argumentos, que puede ser útil cuando se desea convencer a la función `print()` de que cambie su comportamiento un poco.

No se va a explicar en profundidad ahora. Se planea hacer esto cuando se trate el tema de funciones. Por ahora, simplemente queremos mostrarte como funciona. Siéntete libre de utilizarlo en tus propios programas.

El mecanismo se llama **argumentos de palabra clave**. El nombre se deriva del hecho de que el significado de estos argumentos no se toma de su ubicación (posición) sino de la palabra especial (palabra clave) utilizada para identificarlos.

La función `print()` tiene dos argumentos de palabra clave que se pueden utilizar para estos propósitos. El primero de ellos se llama `end`.

En la ventana del editor se puede ver un ejemplo muy simple de como utilizar un argumento de palabra clave.

Para utilizarlo es necesario conocer algunas reglas:

- Un argumento de palabra clave consta de tres elementos: una **palabra clave** que identifica el argumento (`end` - termina aquí); un **signo de igual** (`=`); y un **valor** asignado a ese argumento

```
1 print("Mi nombre es", "Python.", end=" ")
2 print("Monty Python.")
3
```

Console >_

```
Mi nombre es Python. Monty Python.
```

MODULE (22%)

SECTION (76%)



Sandbox

La función `print()` - los argumentos de palabra clave

Y ahora, es el momento de intentar algo más difícil.

Si observas detenidamente, verás que hemos utilizado el argumento `end`, pero su cadena asignada está vacía (no contiene ningún carácter).

¿Qué pasará ahora? Ejecuta el programa en el editor para averiguarlo.

Ya que al argumento `end` se le ha asignado a nada, la función `print()` tampoco genera nada, una vez que se hayan agotado los argumentos posicionales.

La consola ahora debería mostrar el siguiente texto:

```
Mi nombre es Monty Python.
```

salida

Nota: no se han enviado nuevas líneas a la salida.

La cadena asignada al argumento de la palabra clave `end` puede ser de cualquier longitud. Experimenta con ello si gustas.

```
1 print("Mi nombre es ", end="")
2 print("Monty Python.")
3
```

Console >_

```
Mi nombre es Monty Python.
```

MODULE (24%)

SECTION (81%)



Sandbox

La función `print()` - los argumentos de palabra clave

Se estableció anteriormente que la función `print()` separa los argumentos generados con espacios. Este comportamiento también puede ser cambiado.

El **argumento de palabra clave** que puede hacer esto se denomina `sep` (*separador*).

Observa el código en el editor y ejecútalo.

El argumento `sep` entrega el siguiente resultado:

Mi-nombre-es-Monty-Python.

salida

La función `print()` ahora utiliza un guion, en lugar de un espacio, para separar los argumentos generados.

Nota: el valor del argumento `sep` también puede ser una cadena vacía. Pruébalo tu mismo.

```
1 print("Mi", "nombre", "es", "Monty", "Python.", sep="-")
2
```

Console >_

```
Mi-nombre-es-Monty-Python.
```

MODULE (24%)

SECTION (81%)



Sandbox

La función `print()` - los argumentos de palabra clave

Se estableció anteriormente que la función `print()` separa los argumentos generados con espacios. Este comportamiento también puede ser cambiado.

El **argumento de palabra clave** que puede hacer esto se denomina `sep` (*separador*).

Observa el código en el editor y ejecútalo.

El argumento `sep` entrega el siguiente resultado:

Mi-nombre-es-Monty-Python.

salida

La función `print()` ahora utiliza un guion, en lugar de un espacio, para separar los argumentos generados.

Nota: el valor del argumento `sep` también puede ser una cadena vacía. Pruébalo tu mismo.

```
1 print("Mi", "nombre", "es", "Monty", "Python.", sep="-")
2 print("Carlos", "Antonio", sep="-")
3
```

Console >_

```
Mi-nombre-es-Monty-Python.
Mi-nombre-es-Monty-Python.
Carlos-Antonio
```

MODULE (25%)

SECTION (86%)



Sandbox

La función `print()` - los argumentos de palabra clave

Ambos argumentos de palabras clave pueden **mezclarse en una invocación**, como aquí en la ventana del editor.

El ejemplo no tiene mucho sentido, pero representa visiblemente las interacciones entre `end` y `sep`.

¿Puedes predecir la salida?

Ejecuta el código y ve si coincide con tus predicciones.

Ahora que comprendes la función `print()`, estás listo para aprender cómo almacenar y procesar datos en Python.

Sin `print()`, no se podría ver ningún resultado.

```
1 print("Mi", "nombre", "es", sep=" ", end="")
2 print("Monty", "Python.", sep="*", end="*\n")
3
```

Console >_

```
Mi_nombre_es*Monty*Python.*
```


Python INSTITUTE

< 2.1.1.18 LABORATORIO: La función print() >

MODULE (26%)

SECTION (50%)





5-10 minutos

Nivel de Dificultad

Muy fácil

Objetivos

- Familiarizarse con la función de `print()` y sus capacidades de formato.
- Experimentar con el código de Python.

Escenario

Modifica la primera línea de código en el editor, utilizando las palabras clave `sep` y `end`, para que coincida con el resultado esperado. Recuerda, utilizar dos funciones `print()`.

No cambies nada en la segunda invocación de `print()`.

Salida Esperada

```
Fundamentos***Programación***en...Python
```

Sandbox

```

1 print("Fundamentos","Programación","en", sep="***", end="...")
2 print("Python")
3

```

Console >_

```

File "main.py", line 1
print("Fundamentos","Programación","en" sep="***", end="")
      ^
SyntaxError: invalid syntax

File "main.py", line 1
print("Fundamentos","Programación","en" sep="", end="")
      ^
SyntaxError: invalid syntax

Fundamentos*Programación*enPython
Fundamentos***...Programación***...enPython
Fundamentos***Programación***enPython
Fundamentos***Programación***en...Python

```

2.1.1.19 LABORATORIO: Dando formato a la salida

MODULE (28%)

SECTION (95%)

parte del código, pero hay una condición: aprende de tus errores y saca tus propias conclusiones.

Intenta:

- Minimizar el número de invocaciones de la función `print()` insertando la secuencia `\n` en las cadenas.
- Hacer la flecha dos veces más grande (pero mantener las proporciones).
- Duplicar la flecha, colocando ambas flechas lado a lado; nota: una cadena se puede multiplicar usando el siguiente truco: `"cadena" * 2` producirá `"cadenacadena"` (te contaremos más sobre ello pronto).
- Elimina cualquiera de las comillas y observa detenidamente la respuesta de Python; presta atención a donde Python ve un error: ¿es el lugar en donde realmente existe el error?
- Haz lo mismo con algunos de los paréntesis.
- Cambia cualquiera de las palabras `print` en otra cosa (por ejemplo de minúscula a mayúscula, `Print`) - ¿Qué sucede ahora?
- Reemplaza algunas de las comillas por apóstrofes; observa lo que pasa detenidamente.

```

1 print("      *\n")
2 print("     * *\n")
3 print("    * *\n")
4 print("   * *\n")
5 print("  ***\n")
6 print(" * *\n")
7 print(" * *\n")
8 print(" *****\n")
9 print("      *\n      * *\n      * *\n      * *\n      * *\n")
10
11 print("          *")
12 print("         * *")
13 print("        * *")
14 print("       * *")
15 print("      * *")
16 print("     * *")
17 print("    * *")
18 print("   * *")
19 print("*****")
20 print(" * *")
21 print(" * *")
22 print(" * *")
23 print(" * *")
24 print("*****")
25
26 Print(' * * * 2)

```

Console >_

```

*      *
*      *
*      *
*****

```

Traceback (most recent call last):

[illegible]

MODULE (32%)

SECTION (17%)



Sandbox

Literales - los datos en si mismos

Comencemos con un sencillo experimento, observa el fragmento de código en el editor.

La primera línea luce familiar. La segunda parece ser errónea debido a la falta visible de comillas.

Intenta ejecutarlo.

Si todo salió bien, ahora deberías de ver dos líneas idénticas.

¿Qué paso? ¿Qué significa?

A través de este ejemplo, encuentras dos tipos diferentes de literales:

- Una **cadena**, la cual ya conoces.
- Y un número **entero**, algo completamente nuevo.

La función `print()` los muestra exactamente de la misma manera. Sin embargo, internamente, la memoria de la computadora los almacena de dos maneras completamente diferentes. La cadena existe como eso, solo una cadena, una serie de letras.

El número es convertido a una representación máquina (una serie de bits). La función

```
1 print("2")
2 print(2)
3
```

Console >_

```
2
2
```

MODULE (35%)

SECTION (33%)



Sandbox

Utilizar números octales y hexadecimales

Existen dos convenciones adicionales en Python que no son conocidas en el mundo de las matemáticas. El primero nos permite utilizar un número en su representación **octal**.

Si un número entero esta precedido por un código `0o` o `0O` (cero-o), el número será tratado como un valor octal. Esto significa que el número debe contener dígitos en el rango del [0..7] únicamente.

`0o123` es un número **octal** con un valor (decimal) igual a `83`.

La función `print()` realiza la conversión automáticamente. Intenta esto:

```
print(0o123)
```

La segunda convención nos permite utilizar números en **hexadecimal**. Dichos números deben ser precedidos por el prefijo `0x` o `0X` (cero-x).

`0x123` es un número **hexadecimal** con un valor (decimal) igual a `291`. La función `print()` puede manejar estos valores también. Intenta esto:

```
print(0x123)
```

```
1 print(0o123)
2 print(0x123)
3
```

Console >_

```
83
83
291
```

MODULE (43%)

SECTION (75%)



Sandbox

```
I'm Monty Python.
```

¿Sabes cómo hacerlo? Haz clic en [Revisar](#) para saber si estas en lo cierto:

[Revisar](#)

```
print('I\'m Monty Python.')
```

```
o
```

```
print("I'm Monty Python.")
```

Como se puede observar, la diagonal invertida es una herramienta muy poderosa, puede escapar no solo comillas, sino también apóstrofes.

Ya se ha mostrado, pero se desea hacer énfasis en este fenómeno una vez mas - **una cadena puede estar vacía** - puede no contener carácter alguno.

Una cadena vacía sigue siendo una cadena:

```
''
''
```

```
1
2
3 print('I\'m Monty Python.')
4 print("I'm Monty Python.")
5
```

Console >_

```
I'm Monty Python.
I'm Monty Python.
I'm Monty Python.
```

MODULE (44%)

SECTION (92%)



Sandbox

5-10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con la función `print()` y sus capacidades de formato.
- Practicar el codificar cadenas.
- Experimentar con el código de Python.

Escenario

Escribe una sola línea de código, utilizando la función `print()`, así como los caracteres de nueva línea y escape, para obtener la salida esperada de tres líneas.

Salida Esperada

```
"Estoy"
"aprendiendo"
"Python"
```

salida

```
1 print("\n\"Estoy\\n\\\"aprendiendo\\n\\\"Python\\n\\\"")
```

Console >_

```
"Estoy"
"aprendiendo"
"Python"
```

MODULE (47%)

SECTION (10%)



Sandbox

Los operadores básicos

Un **operador** es un símbolo del lenguaje de programación, el cual es capaz de realizar operaciones con los valores.

Por ejemplo, como en la aritmética, el signo de `+` (más) es un operador el cual es capaz de **sumar** dos números, dando el resultado de la suma.

Sin embargo, no todos los operadores de Python son tan simples como el signo de más, veamos algunos de los operadores disponibles en Python, las reglas que se deben seguir para emplearlos, y como interpretar las reglas que realizan.

Se comenzará con los operadores que están asociados con las operaciones aritméticas más conocidas:

`+`, `-`, `*`, `/`, `//`, `%`, `**`

El orden en el que aparecen no es por casualidad. Hablaremos más de ello cuando se hayan visto todos.

Recuerda: Cuando los datos y operadores se unen, forman juntos **expresiones**. La expresión más sencilla es el literal.

```
1 print(2+2)
```

Console >_

4

MODULE (49%)

SECTION (20%)



Sandbox

2³ Los editores de texto puros no aceptan esa notación, por lo tanto Python utiliza `**` en lugar de la notación matemática, por ejemplo, `2 ** 3`.

Observa los ejemplos en la ventana del editor.

Nota: En los ejemplos, los dobles asteriscos están rodeados de espacios, no es obligatorio hacerlo pero hace que el código sea más **legible**.

Los ejemplos muestran una característica importante de los **operadores numéricos** de Python.

Ejecuta el código y observa cuidadosamente los resultados que arroja. ¿Puedes observar algo?

Recuerda: Es posible formular las siguientes reglas con base en los resultados:

- Cuando **ambos** `**` argumentos son enteros, el resultado es entero también.
- Cuando **al menos un** `**` argumento es flotante, el resultado también es flotante.

Esta es una distinción importante que se debe recordar.

```
1 print(2 ** 3)
2 print(2 ** 3.)
3 print(2., ** 3)
4 print(2., ** 3.)
5
```

Console >_

```
8
8.0
8.0
8.0
```

MODULE (50%)

SECTION (30%)



Sandbox

Operadores Aritméticos: división

Un símbolo de `/` (diagonal) es un operador de **división**.

El valor antes de la diagonal es el **dividendo**, y el valor después de la diagonal es el **divisor**.

Ejecuta el código y analiza los resultados.

```
print(6 / 3)
print(6 / 3.)
print(6. / 3)
print(6. / 3.)
```

Deberías de poder observar que hay una excepción a la regla.

El resultado producido por el operador de división siempre es flotante, sin importar si a primera vista el resultado es flotante: `1 / 2`, o si parece ser completamente entero: `2 / 1`.

¿Esto ocasiona un problema? Sí, en ocasiones se podrá necesitar que el resultado de una división sea entero, no flotante.

```
1 print(2 * 3)
2 print(2 * 3.)
3 print(2. * 3)
4 print(2. * 3.)
5
6 print(6 / 3)
7 print(6 / 3.)
8 print(6. / 3)
9 print(6. / 3.)
10
11
```

Console >_

```
6
6.0
6.0
6.0
6
6.0
6.0
6.0
2.0
2.0
2.0
```

MODULE (51%)

SECTION (40%)



Sandbox

Operadores Aritméticos: división entera

Un símbolo de `//` (doble diagonal) es un operador de **división entera**. Difiere del operador estándar `/` en dos detalles:

- El resultado carece de la parte fraccionaria, está ausente (para los enteros), o siempre es igual a cero (para los flotantes); esto significa que **los resultados siempre son redondeados**.
- Se ajusta a la regla *entero frente a flotante*.

Ejecuta el ejemplo debajo y observa los resultados:

```
print(6 // 3)
print(6 // 3.)
print(6. // 3)
print(6. // 3.)
```

Como se puede observar, una división de entero entre entero da un **resultado entero**. Todos los demás casos producen flotantes.

Hagamos algunas pruebas mas avanzadas.

Observa el siguiente fragmento de código:

```
1 print(6 // 3)
2 print(6 // 3.)
3 print(6. // 3)
4 print(6. // 3.)
5
```

Console >_

```
2
2.0
2.0
2.0
```



Sandbox

manzanas, María tenía cinco manzanas, y Adán tenía seis manzanas. Todos eran muy felices y vivieron por muchísimo tiempo. Fin de la Historia.

Tu tarea es:

- Crear las variables: `juan`, `maria`, y `adan`.
- Asignar valores a las variables. El valor debe de ser igual al número de manzanas que cada quien tenía.
- Una vez almacenados los números en las variables, imprimir las variables en una línea, y separar cada una de ellas con una coma.
- Después se debe crear una nueva variable llamada `total_manzanas` y se debe igualar a la suma de las tres variables anteriores.
- Imprime el valor almacenado en `total_manzanas` en la consola.
- Experimenta con tu código:** crea nuevas variables, asigna diferentes valores a ellas, y realiza varias operaciones aritméticas con ellas (por ejemplo, `+`, `-`, `*`, `/`, `//`, etc.). Intenta poner una cadena con un entero juntos en la misma línea, por ejemplo, "Número Total de Manzanas:" y `total_manzanas`.

```
6 total_manzanas = juan + maria + adan
7 print("Total de manzanas", total_manzanas)
8
9
10 naranjas = 20
11 fresa = 10
12 manzanas = 5
13 print(naranjas, fresa, manzanas)
14
15 multiplicacion_frutas = naranjas * fresa * manzanas
16 print("Resultado de multiplicacion de frutas es; ",multiplicacion_fruta
17
18 naranjas_juan = 10
19 naranjas_maria = 4
20
21 suma_naranjas = naranjas_juan + naranjas_maria
22 resta_naranjas = naranjas_juan - naranjas_maria
23 multiplicacion_naranjas = naranjas_maria * 2
24 division_naranjas = naranjas_juan / naranjas_maria
25 division_entera = naranjas_juan // naranjas_maria
26
27 print("Suma de naranjas:", suma_naranjas)
28 print("Resta de naranjas:", resta_naranjas)
29 print("Multiplicación:", multiplicacion_naranjas)
30 print("División (/):", division_naranjas)
31 print("División entera (//):", division_entera)
```

Console >_

```
NameError: name 'limones' is not defined
1, 2, 2
Total de manzanas 5
20 10 5
Resultado de multiplicacion de frutas es; 2000
```

MODULE (72%)

SECTION (82%)

☰

🔍

⚙️

▶️

📄

🔗

📥

📂

🔄

Sandbox

preocupes si no todo queda muy claro. Solo se quiere impulsar tu curiosidad.

Después de completar el laboratorio, abre Sandbox, y experimenta más. Intenta escribir diferentes convertidores, por ejemplo, un convertidor de USD a EUR, un convertidor de temperatura, etc. ¡Deja que tu imaginación vuele! Intenta mostrar los resultados combinando cadenas y variables. Intenta utilizar y experimentar con la función `round()` para redondear tus resultados a uno, dos o tres decimales. Revisa que es lo que sucede si no se provee un dígito al redondear. Recuerda probar tus programas.

Experimenta, saca tus propias conclusiones, y aprende. Sé curioso.

Resultado Esperado

7.38 millas son 11.88 kilómetros
12.25 kilómetros son 7.61 millas

salida

```
1 kilometers = 12.25
2 miles = 7.38
3
4 miles_to_kilometers = miles * 1.61
5 kilometers_to_miles = kilometers / 1.61
6
7 print(miles, "millas son", round(miles_to_kilometers, 2), "kilómetros")
8 print(kilometers, "kilómetros son", round(kilometers_to_miles, 2), "mi
9
```

Console >_

```
^
SyntaxError: invalid syntax
Traceback (most recent call last):
  File "main.py", line 5, in <module>
    kilometers_to_miles = kilometers / 1.61
NameError: name 'kilometers' is not defined
7.38 millas son 11.88 kilómetros
12.25 kilómetros son 7.61 millas
```

MODULE (74%)

SECTION (91%)

☰

🔍

⚙️

▶️

📄

🔗

📥

📂

🔄

Sandbox

Recomiendo que si notaron algunos errores muy seguros omite el operador de multiplicación, aquí se debe de incluir de manera explícita. Nota como se cambia el tipo de dato para asegurarnos de que `x` es del tipo `flotante`.

Mantén tu código limpio y legible, y pruébalo utilizando los datos que han sido proporcionados. No te desanimes por no lograrlo en el primer intento. Se persistente y curioso.

Datos de Prueba

Entrada de Muestra

x = 0
x = 1
x = -1

salida

Salida Esperada

y = -1.0
y = 3.0
y = -9.0

salida

```
1 x = 1
2 x = float(x)
3 # escribe tu código aquí
4 y= 3 * x**3 -2 * x**2 +3 * x -1
5 print("y =", y)
6
```

Console >_

```
y = -1.0
y = 3.0
y = 3.0
```

MODULE (78%)

SECTION (67%)

Sandbox

código.

NOTA

Los comentarios son muy importantes. No solo hacen que el programa sea **más fácil de entender**, pero también sirven para **deshabilitar aquellas partes de código que no son necesarias** (por ejemplo, cuando se necesita probar cierta parte del código, e ignorar el resto). Los buenos programadores **describen** cada parte importante del código, y dan **nombres significativos** a variables, debido a que en ocasiones es mucho más sencillo dejar el comentario dentro del código mismo.

Es bueno utilizar nombres de variables **legibles**, y en ocasiones es mejor **dividir el código** en partes con nombres (por ejemplo en funciones). En algunas situaciones, es una buena idea escribir los pasos de como se realizaron los cálculos de una forma sencilla y clara.

Una cosa mas: puede ocurrir que un comentario contenga una pieza de información incorrecta o errónea, nunca se debe de hacer eso a propósito.

```
1 # este programa calcula los segundos en cierto número de horas determi
2 # este programa fue escrito hace dos días
3
4 a = 2 # número de horas
5 seconds = 3600 # número de segundos en una hora
6
7 print("Horas: ", a) #imprime el numero de horas
8 print("Segundos en Horas: ", a * seconds) # se imprime el numero de se
9
10 #aquí también se debe de imprimir un "Adiós", pero el programador no t
11 #este el es fin del programa que calcula el numero de segundos en 2 hc
12
```

Console >_

Horas: 2
Segundos en Horas: 7200

MODULE (92%)

SECTION (89%)

Sandbox

5-10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con la entrada y salida de datos en Python.
- Evaluar expresiones simples.

Escenario

La tarea es completar el código para evaluar y mostrar el resultado de cuatro operaciones aritméticas básicas.

El resultado debe ser mostrado en consola.

Quizá no podrás proteger el código de un usuario que intente dividir entre cero. Por ahora, no hay que preocuparse por ello.

Prueba tu código - ¿Produce los resultados esperados?

No te mostraremos ningún dato de prueba, eso sería demasiado sencillo.

```
1 a= float(input("Ingrese el primer numero: "))
2 b=float(input("Ingrese el segundo numero: "))
3
4 suma= a + b
5 resta= a - b
6 multiplicacion= a / b
7 potencia= a **2, b **2
8 print("El resultado es", suma)
9 print("El resultado es", resta);
10 print("El resultado es", multiplicacion)
11
12
13 # muestra el resultado de la suma aqui
14 # muestra el resultado de la resta aqui
15 # muestra el resultado de la multiplicación aqui
16 # muestra el resultado de la división aqui
17
18 print("\n¡Eso es todo, amigos!")
19
```

Console >_

Ingrese el primer numero: 10
Ingrese el segundo numero: 20
El resultado es 30.0
El resultado es -10.0
El resultado es 0.5

¡Eso es todo, amigos!

MODULE (93%)

SECTION (77%)

Sandbox

Entrada de muestra: 1

Salida esperada:

y = 0.6000000000000001

Entrada de muestra: 10

Salida esperada:

y = 0.09901951266867294

Entrada de muestra: 100

Salida esperada:

y = 0.009999000199950014

Entrada de muestra: -5

Salida esperada:

y = -0.19258202567760344

```
1 x = float(input("Ingresa el valor para x: "))
2
3 y= 1 / ((x + 1 / (x + 1 / (x + 1 / x))))
4 print("y =", y)
5
```

Console >_

Ingrese el valor para x: 1
y = 0.6000000000000001
Ingrese el valor para x: 10
y = 0.09901951266867294
Ingrese el valor para x: 100
y = 0.009999000199950014
Ingrese el valor para x: -5
y = -0.19258202567760344

MODULE (94%)

SECTION (85%)



Sandbox

Entrada de muestra:

12
17
59

Salida esperada: 13:16

Entrada de muestra:

23
58
642

Salida esperada: 10:40

Entrada de muestra:

0
1
2939

Salida esperada: 1:0

```
1 hour = int(input("Hora de inicio (horas): "))
2 mins = int(input("Minuto de inicio (minutos): "))
3 dura = int(input("Duración del evento (minutos): "))
4
5 total_mins = mins + dura
6 extra_hours = total_mins // 60
7 final_mins = total_mins % 60
8
9 total_hours = hour + extra_hours
10 final_hours = total_hours % 24
11
12 print(f"{final_hours}:{final_mins}")
13
14
```

Console >_

```
Hora de inicio (horas): 12
Minuto de inicio (minutos): 17
Duración del evento (minutos): 59
13:16
Hora de inicio (horas): 23
Minuto de inicio (minutos): 58
Duración del evento (minutos): 642
10:40
Hora de inicio (horas): 0
Minuto de inicio (minutos): 1
Duración del evento (minutos): 2939
1:0
```

Progreso (100%)



Tu puntaje: 10/10

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 2 Quiz

100%

Volver a presentar la prueba

Revisar prueba



« FP1: Examen Módulo 2 »

 En línea

Progreso (100%)

Tu puntaje: 20/20

100%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 2 Test	100%
----------------------	------

Volver a presentar la prueba

Revisar prueba

Modulo 3

Python INSTITUTE

« 3.1.1.4 LABORATORIO: Preguntas y respuestas »

HOME

MODULE (7%)

SECTION (33%)

≡

🔍

⚙️

▶️

📄

🔗

👤

📁

🔄

Sandbox

Ejemplo de entrada: 99

Resultado esperado: False

Ejemplo de entrada: 100

Resultado esperado: True

Ejemplo de entrada: 101

Resultado esperado: True

Ejemplo de entrada: -5

Resultado esperado: False

Ejemplo de entrada: +123

Resultado esperado: True

```
1 n = int(input())
2 print(n >= 100)
3
```

Console >_

55
False
99
False
100
True
101
True
-5
False
123
True

Report a Bug

Prueba tu código con los datos que te proporcionamos. ¡Y hazte de un ESPATIFILIO también!

Datos de Prueba

Entrada de muestra: espatifilo

Resultado esperado: No, ¡quiero un gran ESPATIFILIO!

Entrada de ejemplo: pelargonio

Resultado esperado: ¡ESPATIFILIO!, ¡No pelargonio!

Entrada de muestra: ESPATIFILIO

Resultado esperado: Si, ¡El ESPATIFILIO es la mejor planta de todos los tiempos!

```
1
2 entrada = input("Ingresa el nombre de una planta: ")
3
4 if entrada == "ESPATIFILIO":
5     print("Si, ¡El ESPATIFILIO es la mejor planta de todos los tiempos!")
6 elif entrada == "espatifilo":
7     print("No, ¡quiero un gran ESPATIFILIO!")
8 else:
9     print(f"¡ESPATIFILIO!, ¡No {entrada}!")
```

Console >_

Ingresa el nombre de una planta: espatifilo
No, ¡quiero un gran ESPATIFILIO!
Ingresa el nombre de una planta: pelargonio
¡ESPATIFILIO!, ¡No pelargonio!
Ingresa el nombre de una planta: ESPATIFILIO
Si, ¡El ESPATIFILIO es la mejor planta de todos los tiempos!

Python INSTITUTE

« 3.2.1.3 LABORATORIO: Lo esencial del bucle while - Adivina el número secreto »

HOME

MODULE (24%)

SECTION (18%)

≡

🔍

⚙️

▶️

📄

🔗

👤

📁

🔄

Sandbox

Tu tarea es ayudar al mago a completar el código en el editor de tal manera que el código:

- Pedirá al usuario que ingrese un número entero.
- Utilizará un bucle `while`.
- Comprobará si el número ingresado por el usuario es el mismo que el número escogido por el mago. Si el número elegido por el usuario es diferente al número secreto del mago, el usuario debería ver el mensaje: "¡Ja, ja! ¡Estás atrapado en mi bucle!" y se le solicitará que ingrese un número nuevamente. Si el número ingresado por el usuario coincide con el número escogido por el mago, el número debe imprimirse en la pantalla, y el mago debe decir las siguientes palabras: "¡Bien hecho, muggle! Eres libre ahora".

¡El mago está contando contigo! No lo decepciones.

INFO EXTRA

Por cierto, observa la función `print()`. La forma en que lo hemos utilizado aquí se llama *impresión multilinea*. Puede utilizar *comillas triples* para imprimir cadenas en varias líneas para facilitar la lectura del texto o crear un diseño especial basado en texto. Experimenta con ello.

```
9 | elegido para ti. |
10 | Entonces, |
11 | ¿Cuál es el número secreto? |
12 | +-----+ |
13 | """ |
14 | |
15 | number = int(input("Introduce un número entero: ")) |
16 | |
17 | |
18 | while number != secret_number: |
19 |     print("¡Ja, ja! ¡Estás atrapado en mi bucle!") |
20 |     number = int(input("Inténtalo de nuevo, introduce un número entero: ")) |
21 | |
22 | |
23 | print(number)
```

Console >_

¡Bien hecho, muggle! Eres libre ahora

+-----+
| ¡Bienvenido a mi juego, muggle! |
| Introduce un número entero |
| y adivina qué número he |
| elegido para ti. |
| Entonces, |
| ¿Cuál es el número secreto? |
+-----+

Introduce un número entero: 777
777
¡Bien hecho, muggle! Eres libre ahora

MODULE (16%)

SECTION (80%)

Menu

Icons

Settings

▶

📄

🔗

📄

📄

🔄

Sandbox

algunos cálculos inteligentes.

Prueba tu código con los datos que hemos proporcionado.

Datos de Prueba

Entrada de muestra:

Resultado esperado:

Entrada de muestra:

Resultado esperado:

Entrada de muestra:

Resultado esperado:

Entrada de muestra:

Resultado esperado:

```
1 income = float(input("Introduce el ingreso anual:"))
2
3 # Escribe tu código aquí.
4 if income <= 85528:
5     tax = income * 0.18 - 556.02
6 else:
7     tax = 14839.02 + (income - 85528) * 0.32
8
9 if tax < 0:
10     tax = 0.0
11
12 tax = round(tax, 0)
13 print("El impuesto es:", tax, "pesos")
```

Console >_

↻

Introduce el ingreso anual:10000
El impuesto es: 1244.0 pesos
Introduce el ingreso anual:100000
El impuesto es: 19470.0 pesos
Introduce el ingreso anual:1000
El impuesto es: 0.0 pesos
Introduce el ingreso anual:-100
El impuesto es: 0.0 pesos

MODULE (17%)

SECTION (87%)

Menu

Icons

Settings

▶

📄

🔗

📄

📄

🔄

Sandbox

Introduce un año:

Resultado esperado:

Entrada de muestra:

Resultado esperado:

Entrada de muestra:

Resultado esperado:

Entrada de muestra:

Resultado esperado:

Entrada de muestra:

Resultado esperado:

```
1 year = int(input("Introduce un año: "))
2
3 if year < 1582:
4     print("No esta dentro del periodo del calendario Gregoriano")
5 else:
6     if year % 4 != 0:
7         print("Año Común")
8     elif year % 100 != 0:
9         print("Año Bisiesto")
10    elif year % 400 != 0:
11        print("Año Común")
12    else:
13        print("Año Bisiesto")
```

Console >_

↻

Introduce un año: 2000
Año Bisiesto
Introduce un año: 2015
Año Común
Introduce un año: 1999
Año Común
Introduce un año: 1996
Año Bisiesto
Introduce un año: 1580
No esta dentro del periodo del calendario Gregoriano

MODULE (28%)

SECTION (35%)

LABORATORIO

Tiempo Estimado

5 minutos

Nivel de dificultad

Muy fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar el bucle `for`.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

¿Sabes lo que es Mississippi? Bueno, es el nombre de uno de los estados y ríos en los Estados Unidos. El río Mississippi tiene aproximadamente 2.340 millas de largo, lo que lo convierte en el segundo río más largo de los Estados Unidos (el más largo es el río Missouri). ¡Es tan largo que una sola gota de agua necesita 90 días para recorrer toda su longitud!

La palabra *Mississippi* también se usa para un propósito ligeramente diferente: para

```
1 import time
2
3 # Escribe un bucle for que cuente hasta cinco.
4 for i in range(1, 6):
5     print(i, "Mississippi")
6 # Cuerpo del bucle: imprime el número de iteración del bucle y la palabra "Mississippi".
7 # Cuerpo del bucle - usar: time.sleep(1)
8 time.sleep(1)
9
10 # Escribe una función de impresión con el mensaje final.
11 print("¡Listos o no, ahí voy!")
```

Console >_

Sandbox

MODULE (28%)

SECTION (35%)

"contar de forma mississippi" hasta cinco. Habiendo contado hasta cinco, el programa debería imprimir en la pantalla el mensaje final: "¡Listos o no, ahí voy!"

Utiliza el esqueleto que hemos proporcionado en el editor.

INFO EXTRA

Ten en cuenta que el código en el editor contiene dos elementos que pueden no ser del todo claros en este momento: la sentencia `import time` y el método `sleep()`. Vamos a hablar de ellos pronto.

Por el momento, nos gustaría que supieras que hemos importado el módulo `time` y hemos utilizado el método `sleep()` para suspender la ejecución de cada función posterior de `print()` dentro del bucle `for` durante un segundo, de modo que el mensaje enviado a la consola se parezca a un conteo real. No te preocupes, pronto aprenderás más sobre módulos y métodos.

Salida esperada

1 Mississippi
2 Mississippi
3 Mississippi
4 Mississippi
5 Mississippi

```
1 import time
2
3 # Escribe un bucle for que cuente hasta cinco.
4 for i in range(1,6):
5     print(i, "Mississippi")
6 # Cuerpo del bucle: imprime el número de iteración del bucle y la palabra "Mississippi".
7 # Cuerpo del bucle - usar: time.sleep(1)
8 time.sleep(1)
9
10 # Escribe una función de impresión con el mensaje final.
11 print("¡Listos o no, ahí voy!")
```

Console >_

1 Mississippi
2 Mississippi
3 Mississippi
4 Mississippi
5 Mississippi
¡Listos o no, ahí voy!

Sandbox

MODULE (32%)

SECTION (53%)

Objetivos

Familiarizar al estudiante con:

- Utilizar la instrucción `break` en los bucles.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

La instrucción `break` se implementa para salir/terminar un bucle.

Diseña un programa que use un bucle `while` y le pida continuamente al usuario que ingrese una palabra a menos que ingrese "chupacabra" como la palabra de salida secreta, en cuyo caso el mensaje "¡Has dejado el bucle con éxito!". Debe imprimirse en la pantalla y el bucle debe terminar.

No imprimas ninguna de las palabras ingresadas por el usuario. Utiliza el concepto de ejecución condicional y la sentencia `break`.

```
1 while True:
2     palabra = input("Ingresa una palabra: ")
3
4     if palabra == "chupacabra":
5         print("¡Has dejado el bucle con éxito!")
6         break
```

Console >_

Ingresa una palabra: Chivas
Ingresa una palabra: Hola
Ingresa una palabra: chupacabra
¡Has dejado el bucle con éxito!

Sandbox

MODULE (33%)

SECTION (59%)

Sandbox

Salida esperada:

G
R
G
R
Y

Entrada de muestra: abstemious

Salida esperada:

B
S
T
M
S

Entrada de muestra: IOUEA

Salida esperada:

```
3 user_word = input("Ingresa una palabra: ")
4 user_word = user_word.upper()
5
6 for letter in user_word:
7     # Completa el cuerpo del bucle for.
8     if letter == "A":
9         continue
10    elif letter == "E":
11        continue
12    elif letter == "I":
13        continue
14    elif letter == "O":
15        continue
16    elif letter == "U":
17        continue
```

Console >_

```
Ingresa una palabra: Gregory
G
R
G
R
Y
Ingresa una palabra: abstemious
B
S
T
M
S
Ingresa una palabra: IOUEA
```

MODULE (35%)

SECTION (65%)

Sandbox

Prueba tu programa con los datos que le proporcionamos.

Datos de Prueba

Entrada de muestra: Gregory

Salida esperada:

GRGRY

Entrada de muestra: abstemious

Salida esperada:

BSTMS

Entrada de muestra: IOUEA

Salida esperada:

```
2 user_word = user_word.upper()
3 word_without_vowels = ""
4
5 # Indicar al usuario que ingrese una palabra
6 # y asignarla a la variable user_word.
7
8
9 for letter in user_word:
10     if letter == "A":
11         continue
12     elif letter == "E":
13         continue
14     elif letter == "I":
15         continue
16     elif letter == "O":
17         continue
18     elif letter == "U":
19         continue
20     else:
21         word_without_vowels += letter
22
23 print(word_without_vowels)
24 # Completa el cuerpo del bucle.
```

Console >_

```
Ingresa una palabra: Gregory
GRGRY
Ingresa una palabra: abstemious
BSTMS
Ingresa una palabra: IOUEA
```

MODULE (39%)

SECTION (82%)



Sandbox

LABORATORIO

Tiempo Estimado

20-30 minutos

Nivel de Dificultad

Medio

Objetivos

Familiarizar al estudiante con:

- Utilizar el bucle `while`.
- Encontrar la implementación adecuada de reglas definidas verbalmente.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

Escucha esta historia: Un niño y su padre, un programador de computadoras, juegan con bloques de madera. Están construyendo una pirámide.

```
1 blocks = int(input("Ingresa el número de bloques: "))
2
3 #
4 # Escribe tu código aquí.
5 height = 0
6 blocks_in_layer = 1
7
8 while blocks >= blocks_in_layer:
9     blocks -= blocks_in_layer
10    height += 1
11    blocks_in_layer += 1
12 #
13
14 print("La altura de la pirámide:", height)
15
```

Console >_



```
Ingresa el número de bloques: 6
La altura de la pirámide: 3
Ingresa el número de bloques: 20
La altura de la pirámide: 5
Ingresa el número de bloques: 1000
La altura de la pirámide: 44
Ingresa el número de bloques: 2
La altura de la pirámide: 1
```

MODULE (40%)

SECTION (88%)



Sandbox

```
148
74
37
112
56
28
14
7
22
11
34
17
52
26
13
40
20
10
5
16
8
4
2
1
pasos = 62
```

```
1 c0 = int(input("Ingresa un número natural: "))
2
3 pasos = 0
4
5 while c0 != 1:
6     if c0 % 2 == 0:
7         c0 = c0 // 2
8     else:
9         c0 = 3 * c0 + 1
10
11     print(c0)
12     pasos += 1
13
14 print("pasos =", pasos)
```

Console >_



```
4
2
1
pasos = 17
Ingresa un número natural: 16
8
4
2
1
pasos = 4
Ingresa un número natural: 1023
3070
1535
```

MODULE (59%)

SECTION (43%)



Sandbox

Objetivos

Familiarizar al estudiante con:

- Usar instrucciones básicas relacionadas con listas.
- Crear y modificar listas.

Escenario

Había una vez un sombrero. El sombrero no contenía conejo, sino una lista de cinco números: 1, 2, 3, 4 y 5.

Tu tarea es:

- Escribir una línea de código que solicite al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1).
- Escribir una línea de código que elimine el último elemento de la lista (Paso 2).
- Escribir una línea de código que imprima la longitud de la lista existente (Paso 3).

¿Listo para este desafío?

```
1 hat_list = [1, 2, 3, 4, 5] # Esta es una lista existente de números  
2  
3 # Paso 1: escribe una línea de código que solicite al usuario  
4 # reemplazar el número de en medio con un número entero ingresado por  
5 hat_list[2] = int(input("Ingresa un número entero para reemplazar el  
6  
7 # Paso 2: escribe aquí una línea de código que elimine el último elem  
8 del hat_list[-1]  
9  
10 # Paso 3: escribe aquí una línea de código que imprima la longitud de  
11 print("La longitud de la lista es:", len(hat_list))  
12  
13 print(hat_list)
```

Console >_

```
Ingresa un número entero para reemplazar el valor central: 5  
La longitud de la lista es: 4  
[1, 2, 5, 4]
```

MODULE (68%)

SECTION (93%)



Sandbox

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Crear y modificar listas simples.
- Utilizar métodos para modificar listas.

Escenario

Los Beatles fueron uno de los grupos de música más populares de la década de 1960 y la banda más vendida en la historia. Algunas personas los consideran el acto más influyente de la era del rock. De hecho, se incluyeron en la compilación de la revista *Time* de las 100 personas más influyentes del siglo XX.

```
1 beatles = []  
2 print("Paso 1:", beatles)  
3  
4 beatles.append("John Lennon")  
5 beatles.append("Paul McCartney")  
6 beatles.append("George Harrison")  
7 print("Paso 2:", beatles)  
8  
9- for i in range(2):  
10     nuevo_miembro = input("Ingresa el nombre del siguiente miembro: ")  
11     beatles.append(nuevo_miembro)  
12 print("Paso 3:", beatles)  
13  
14 del beatles[-1]  
15 del beatles[-1]  
16 print("Paso 4:", beatles)  
17  
18 beatles.insert(0, "Ringo Starr")  
19 print("Paso 5:", beatles)  
20  
21 print("Los Fav", len(beatles))
```

Console >_

```
Paso 1: []  
Paso 2: ['John Lennon', 'Paul McCartney', 'George Harrison']  
Ingresa el nombre del siguiente miembro: George Harrison  
Ingresa el nombre del siguiente miembro: John Lennon
```

MODULE (87%)

SECTION (90%)



Sandbox

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Indexación de listas.
- Utilizar operadores `in` y `not in`.

Escenario

Imagina una lista: no muy larga ni muy complicada, solo una lista simple que contiene algunos números enteros. Algunos de estos números pueden estar repetidos, y esta es la clave. No queremos ninguna repetición. Queremos que sean eliminados.

Tu tarea es escribir un programa que elimine todas las

```
1 my_list = [1, 2, 4, 4, 1, 4, 2, 6, 2, 9]
2
3 temp_list = []
4
5 for number in my_list:
6     if number not in temp_list:
7         temp_list.append(number)
8
9 my_list = temp_list
10
11 print("La lista con elementos únicos:")
12 print(my_list)
```

Console >_



```
La lista con elementos únicos:
[1, 2, 4, 6, 9]
```

Quiz Modulo 3

Progreso (100%)



Tu puntaje: 10/10

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 3 Quiz

100%

Volver a presentar la prueba

Revisar prueba



« FP1: Examen Módulo 3 »



● En línea

Progreso (100%)

Tu puntaje: 20/20

100%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 3 Test	100%
----------------------	------

Volver a presentar la prueba

Revisar prueba

Modulo 4

Python INSTITUTE

4.3.1.6 LABORATORIO: Un año bisiesto: escribiendo tus propias funciones

MODULE (30%)

SECTION (55%)

≡

🔍

⚙️

▶️

📄

🔗

📥

📂

🔄

Sandbox

Fácil

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la instrucción `return`.
- Probar las funciones.

Escenario

Tu tarea es escribir y probar una función que toma un argumento (un año) y devuelve `True` si el año es un *año bisiesto*, o `False` si no lo es.

Parte del esqueleto de la función ya está en el editor.

Nota: también hemos preparado un breve código de prueba, que puedes utilizar para probar tu función.

El código utiliza dos listas: una con los datos de prueba y la otra con los resultados esperados. El código te dirá si alguno de tus resultados no es válido.

```
1 def is_year_leap(year):
2     # Un año es bisiesto si es divisible por 4,
3     # excepto si es divisible por 100, a menos que también sea divisib
4     if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
5         return True
6     return False
7
8 test_data = [1900, 2000, 2016, 1987]
9 test_results = [False, True, True, False]
10 for i in range(len(test_data)):
11     yr = test_data[i]
12     print(yr, "->", end="")
13     result = is_year_leap(yr)
14     if result == test_results[i]:
15         print("OK")
16     else:
17         print("Fallido")
```

Console >_

1900 ->OK
2000 ->OK
2016 ->OK
1987 ->OK

Python INSTITUTE

4.3.1.7 LABORATORIO: ¿Cuántos días?: escribiendo y utilizando tus propias funciones

MODULE (32%)

SECTION (64%)

≡

🔍

⚙️

▶️

📄

🔗

📥

📂

🔄

Sandbox

Objetivos

- Proyectar y escribir funciones parametrizadas.
- Utilizar la instrucción `return`.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que toma dos argumentos (un año y un mes) y devuelve el número de días del mes/año dado (mientras que solo febrero es sensible al valor `year`, tu función debería ser universal).

La parte inicial de la función está lista. Ahora, haz que la función devuelva `None` si los argumentos no tienen sentido.

Por supuesto, puedes (y debes) utilizar la función previamente escrita y probada (LABORATORIO 4.1.3.6). Puede ser muy útil. Te recomendamos que utilices una lista con los meses. Puedes crearla dentro de la función; este truco acortará significativamente el código.

Hemos preparado un código de prueba. Amplíalo para incluir más casos de prueba.

```
4 return False
5
6 def days_in_month(year, month):
7     if month < 1 or month > 12 or year < 1:
8         return None
9
10    days_per_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
11
12    if month == 2 and is_year_leap(year):
13        return 29
14
15    return days_per_month[month - 1]
16
17 test_years = [1900, 2000, 2016, 1987, 2024, 2023, 2023]
18 test_months = [2, 2, 1, 11, 2, 2, 13]
19 test_results = [28, 29, 31, 30, 29, 28, None]
20
21 for i in range(len(test_years)):
22     yr = test_years[i]
23     mo = test_months[i]
24     print(yr, mo, "->", end="")
25     result = days_in_month(yr, mo)
26     if result == test_results[i]:
27         print("OK")
28     else:
29         print(f"Fallido (Obtenido: {result}, Esperado: {test_results[i]})")
```

Console >_

2000 2 ->OK
2016 1 ->OK
1987 11 ->OK
2024 2 ->OK
2023 2 ->OK

MODULE (33%)

SECTION (73%)

LABORATORIO

Tiempo Estimado

20-30 minutos

Nivel de Dificultad

Medio

Requisitos Previos

LABORATORIO 4.1.3.6
LABORATORIO 4.1.3.7

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la sentencia `return`.
- Construir un conjunto de funciones de utilidad.
- Utilizar las funciones propias del estudiante.

Escenario

```
1 def is_year_leap(year):
2     if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
3         return True
4     return False
5
6 def days_in_month(year, month):
7     if month < 1 or month > 12 or year < 1:
8         return None
9
10    days_per_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
11
12    if month == 2 and is_year_leap(year):
13        return 29
14
15    return days_per_month[month - 1]
16
17 test_years = [1900, 2000, 2016, 1987, 2024, 2023, 2023]
```

Console >_

```
1900 2 ->OK
2000 2 ->OK
2016 1 ->OK
1987 11 ->OK
2024 2 ->OK
2023 2 ->OK
2023 13 ->OK
```

MODULE (35%)

SECTION (82%)

- Toma un argumento (el valor a verificar).
- Devuelve `True` si el argumento es un número primo, y `False` de lo contrario.

Sugerencia: intenta dividir el argumento por todos los valores posteriores (comenzando desde 2) y verifica el resto: si es cero, tu número no puede ser un número primo; analiza cuidadosamente cuándo deberías detener el proceso.

Si necesitas conocer la raíz cuadrada de cualquier valor, puedes utilizar el operador `**`. Recuerda: la raíz cuadrada de x es lo mismo que $x^{0.5}$.

Complementa el código en el editor.

Ejecuta tu código y verifica si tu salida es la misma que la nuestra.

Datos de prueba

Salida esperada:

2 3 5 7 11 13 17 19

```
1 def is_prime(num):
2     if num <= 1:
3         return False
4     # Verificamos divisores desde el 2 hasta la raíz cuadrada del número
5     for i in range(2, int(num ** 0.5) + 1):
6         if num % i == 0:
7             return False
8     return True
9
10 for i in range(1, 20):
11     if is_prime(i + 1):
12         print(i + 1, end=" ")
13 print()
```

Console >_

```
2 3 5 7 11 13 17 19
```

MODULE (36%)

SECTION (91%)

- Se llaman `liters_100km_to_miles_gallon` y `miles_gallon_to_liters_100km` respectivamente.
- Toman un argumento (el valor correspondiente a sus nombres).

Complementa el código en el editor.

Ejecuta tu código y verifica si tu salida es la misma que la nuestra.

Aquí hay información para ayudarte:

- 1 milla = 1609.344 metros.
- 1 galón = 3.785411784 litros.

Salida esperada:

```
60.31143162393162
31.361944444444444
23.521458333333333
3.9007393587617467
7.490910297239915
10.009131205673757
```

```
1- def liters_100km_to_miles_gallon(liters):
2-     return 378.5411784 / (liters * 1.609344)
3-
4- def miles_gallon_to_liters_100km(miles):
5-     return 378.5411784 / (miles * 1.609344)
6-
7- print(liters_100km_to_miles_gallon(3.9))
8- print(liters_100km_to_miles_gallon(7.5))
9- print(liters_100km_to_miles_gallon(10.))
10- print(miles_gallon_to_liters_100km(60.3))
11- print(miles_gallon_to_liters_100km(31.4))
12- print(miles_gallon_to_liters_100km(23.5))
```

Console >_

```
60.31143162393162
31.361944444444444
23.521458333333333
3.9007393587617467
7.490910297239915
10.009131205673757
```

MODULE (96%)

SECTION (93%)

- Se llaman `liters_100km_to_miles_gallon` y `miles_gallon_to_liters_100km` respectivamente.
- Toman un argumento (el valor correspondiente a sus nombres).

PROYECTO

Tiempo Estimado

30-60 minutos

Nivel de Dificultad

Medio/Difícil

Objetivos

- Perfeccionar las habilidades del estudiante al emplear Python para resolver problemas complejos.
- La integración de técnicas de programación en un solo programa consistente de varias partes.

Escenario

Tu tarea es escribir un simple programa que simule jugar a *tic-tac-toe* (nombre en inglés) con el usuario. Para hacerlo más fácil, hemos decidido simplificar el juego. Aquí están nuestras reglas:

- La maquina (por ejemplo, el programa) jugará utilizando las 'x' s.

```
1- from random import randrange
2-
3- def DisplayBoard(board):
4-     print("+-----+-----+")
5-     for row in board:
6-         print("|       |       |")
7-         print(f"| {row[0]} | {row[1]} | {row[2]} |")
8-         print("|       |       |")
9-     print("+-----+-----+")
10-
11- def EnterMove(board):
12-     ok = False
13-     while not ok:
14-         try:
15-             move = int(input("Ingresa tu movimiento: "))
```

Console >_

```
| 7 | 0 | 9 |
|
+-----+-----+
+-----+-----+
|   |   |   |
| 1 | 2 | 3 |
|   |   |   |
+-----+-----+
|   |   |   |
|  X |  X | 6 |
|   |   |   |
+-----+-----+
|   |   |   |
|   |   |   |
|   |   |   |
```

Quiz modulo 4



« PE1 -- Module 4 Quiz »

 En línea

Progreso (100%)



Tu puntaje: 12/12

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 4 Quiz100%

Volver a presentar la prueba

Revisar prueba

Test modulo 4



« FP1: Examen Módulo 4 »

 En línea

Progreso (100%)



Tu puntaje: 22/22

100%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 4 Test100%

Volver a presentar la prueba

Revisar prueba

Python Essentials 1 | PCEP-30-02 Ready [Spanish]

Welcome to PE1	PE1: Module 1	PE1: Module 2	PE1: Module 3	PE1: Module 4	PE1 Summary Test	PCEP-30-0x Exam
About the Course ✓	Study Resources ✓ Quiz: 100% Test: 100%	Study Resources ✓ Quiz: 100% Test: 100%	Study Resources ✓ Quiz: 100% Test: 100%	Study Resources ✓ Quiz: 100% Test: 100%	FINAL TEST: -	

Test final

« PE1 - Examen De Sección »

Progreso (100%)

Tu puntaje: 32/35

91%

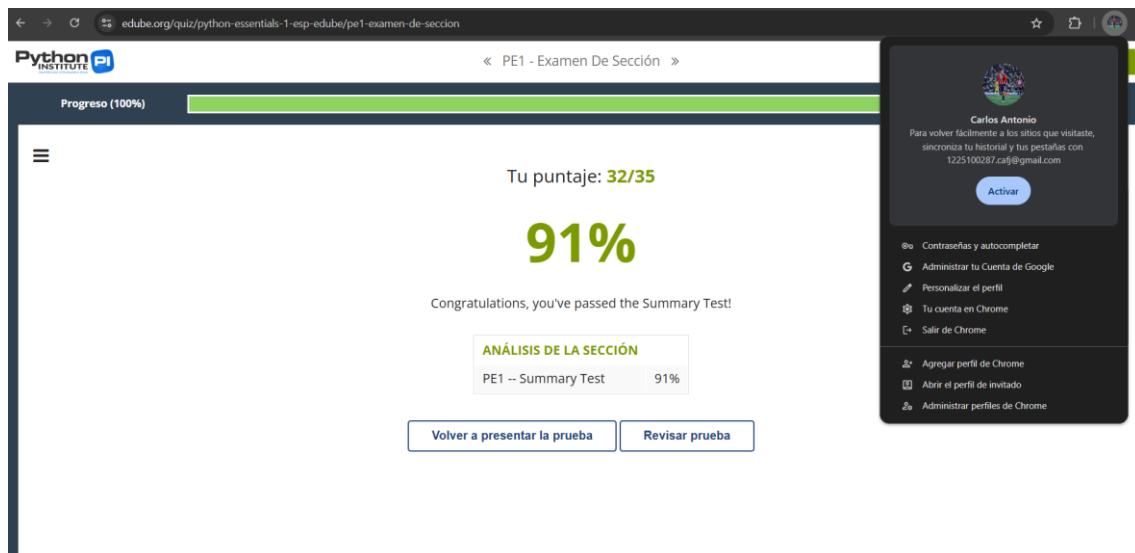
Congratulations, you've passed the Summary Test!

ANÁLISIS DE LA SECCIÓN

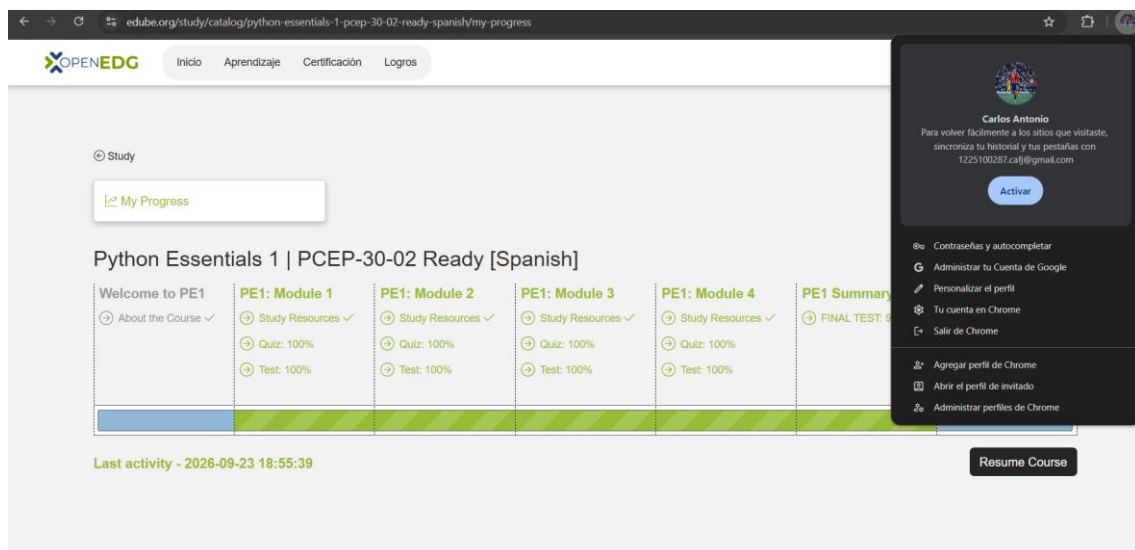
PE1 -- Summary Test	91%
---------------------	-----

Volver a presentar la prueba

Revisar prueba



Captura de finalización de curso



Nota: Toda captura de pantalla de test, quiz y laboratorios son capturas originalmente más, no hubo capturas plagio de los demás compañeros

